

APEXAIQ

**LINUX AND DEVOPS RESEARCH
REPORT**

**Submitted By
Pranay Devidas Hajare (CSE)**

**Shri Sant Gajanan Maharaj College of Engineering,
Shegaon**

What is Linux?

Linux is an open-source, Unix-like operating system that manages computer hardware and software and provides an environment for running applications.

It acts as an interface between the user, applications, and computer hardware.

Linux is widely used in servers, cloud computing, software development, cybersecurity, networking, and DevOps.

1. What is an Operating System?

An **operating system (OS)** is system software that manages computer hardware and software resources and provides services to application programs.

Examples:

- Windows
- Linux
- macOS
- Android

Main functions of an operating system:

1. **Process Management:** Manages the execution of programs and processes.
2. **Memory Management:** Allocates and manages RAM for running programs.
3. **File Management:** Creates, stores, reads, and deletes files and directories.
4. **Device Management:** Manages hardware devices such as keyboards, printers, and disks.
5. **Security:** Controls user access and permissions.
6. **Resource Management:** Allocates CPU, memory, and other resources.

2. What is Linux?

Linux is an open-source operating system based on the Linux kernel, originally created by Linus Torvalds in 1991.

The Linux kernel is the core component of the operating system. It manages hardware resources and allows software to communicate with hardware.

Linux is available in different distributions (distros).

Examples of Linux distributions:

- Ubuntu
- Debian

- Fedora
- Red Hat Enterprise Linux (RHEL)
- Kali Linux
- Linux Mint
- Arch Linux

Each distribution packages the Linux kernel with different software and tools for particular needs.

3. Why Is Linux Used?

Linux is used because it is free, open-source, secure, stable, customizable, and suitable for servers and development environments.

1. Open Source

Linux's source code is publicly available. Developers can study, modify, and distribute it according to its license.

2. Free to Use

Many Linux distributions are available free of charge. Users can install and use them without purchasing an operating system license.

3. Security

Linux provides user permissions, file permissions, and security mechanisms to control access to system resources.

4. Stability

Linux is widely used on servers because it can run continuously for long periods when properly maintained.

5. Multitasking

Linux allows multiple processes to run concurrently, sharing CPU and other system resources.

Basic Linux Commands

1.1 File & Directory Commands

These are the commands you'll type without thinking once they're second nature — navigating around the filesystem, creating and removing things, and looking inside files.

Command	What it does
pwd	Print the full path of the directory you're currently in.
ls / ls -la	List the contents of a directory. Adding -la shows hidden files, permissions, owners and sizes.
cd <path>	Change into a directory. cd .. goes up one level; cd ~ goes to your home directory.
mkdir <name>	Create a new directory. Use mkdir -p a/b/c to create nested directories in one go.
touch <file>	Create an empty file, or update its last-modified timestamp if it already exists.
cp <src> <dst>	Copy a file. Add -r to copy an entire directory recursively.
mv <src> <dst>	Move or rename a file or directory.
rm <file>	Delete a file. rm -r deletes a directory and everything inside it — there's no recycle bin, so double-check the path.
cat <file>	Print an entire file's contents to the screen.
less <file>	Open a file for scrollable, page-by-page reading (press q to quit).
head / tail <file>	Show the first (head) or last (tail) lines of a file. tail -f follows a file live, e.g. a log.
find <path> -name '<pattern>'	Search a directory tree for files matching a name pattern.
grep '<text>' <file>	Search inside a file (or, with -r, a whole directory) for lines containing a pattern.
chmod <mode> <file>	Change a file's permissions, e.g. chmod 755 script.sh makes it executable.
chown <user>:<group> <file>	Change who owns a file and which group it belongs to.
tar -czvf archive.tar.gz <dir>	Compress a directory into a single .tar.gz archive.
tar -xzvf archive.tar.gz	Extract a .tar.gz archive back into its files.

1.2 Process Management Commands

Once things are running on a server — a Docker daemon, a web server, your own script — you need to be able to see what's active, how much of the machine it's using, and how to stop it cleanly (or forcefully) when needed.

Command	What it does
---------	--------------

ps aux	List every process currently running on the system, with its PID (process ID), owner, and resource usage.
top	Show a live, auto-refreshing view of running processes ranked by CPU/memory usage. Press q to exit.
htop	A friendlier, color, scrollable alternative to top (install with <code>sudo apt install htop</code>).
kill <PID>	Ask a process to shut down gracefully by its process ID.
kill -9 <PID>	Force-kill a process immediately when it won't respond to a normal kill.
killall <name>	Kill every process matching a given name, without needing the PID.
<command> &	Run a command in the background so your terminal is free to keep working.
jobs	List the background/suspended jobs running in your current shell session.
fg / bg	Bring a background job to the foreground (fg), or resume a suspended job in the background (bg).
Ctrl+Z	Suspend (pause) the process currently running in the foreground.
nohup <command> &	Run a command so it keeps running even after you log out of the terminal session.
nice -n <value> <command>	Start a process with a lower or higher CPU scheduling priority.
systemctl start/stop/status <service>	Start, stop, or check the status of a system service (e.g. docker, nginx).
systemctl enable <service>	Set a service to start automatically on every system boot.

1.3 Networking & File Transfer

A DevOps engineer is constantly checking connectivity, pulling files down from remote hosts, and pushing them back up.

Command	What it does
ip a	Show the network interfaces on the machine and their assigned IP addresses (modern replacement for ifconfig).
ping <host>	Send test packets to a host to check that it's reachable and measure latency.
curl <url>	Fetch a URL from the command line — great for quickly testing whether a web service is responding.
wget <url>	Download a file from a URL directly to disk.
ss -tulpn	List listening ports and the process bound to each one (modern replacement for netstat).
ssh user@host	Open a secure remote shell session on another machine.
scp <file> user@host:<path>	Securely copy a file to (or from) a remote machine over SSH.

1. What is DevOps?

DevOps (Development + Operations) is a combination of practices, tools, and cultural principles that help development (Dev) and IT operations (Ops) teams work together to develop, test, deploy, and maintain software faster and more reliably.

In traditional software development, developers write code, while operations teams deploy and maintain applications. These teams often work separately, which can cause delays and communication problems.

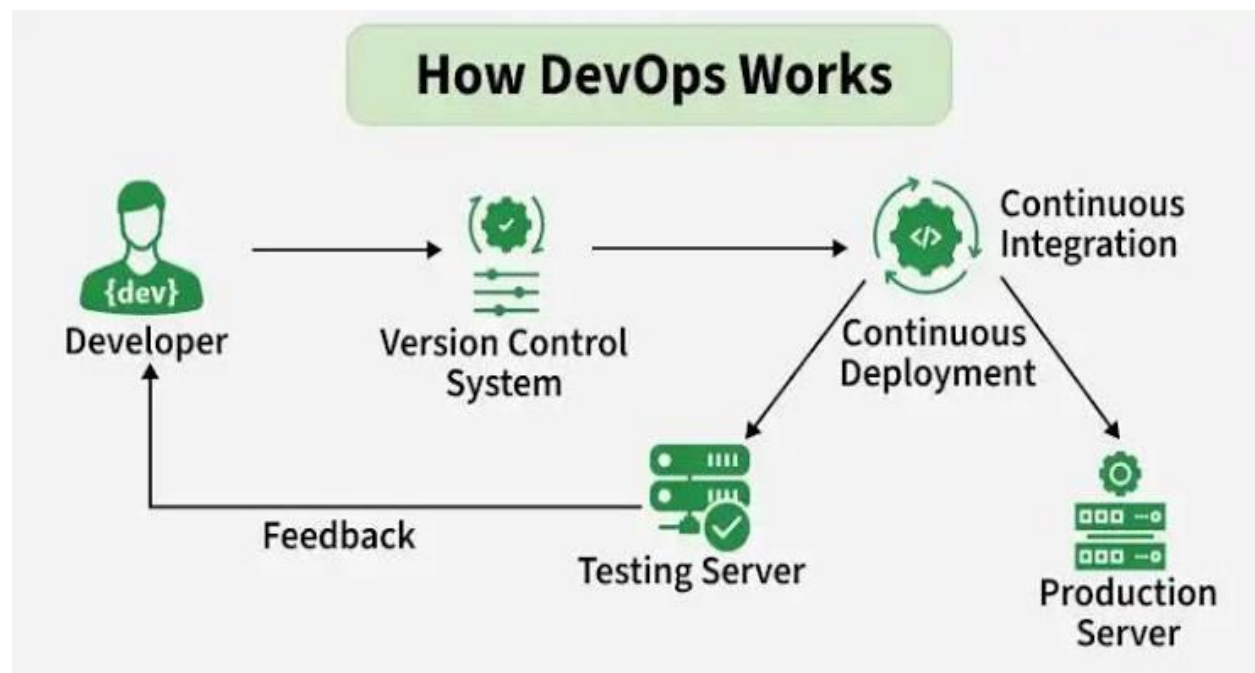
DevOps bridges the gap between these teams through collaboration, automation, and continuous delivery.

Example:

Suppose you develop a restaurant management application using React and Django.

- **Development team:** Writes the application code and fixes bugs.
- **Operations team:** Manages servers, databases, and application deployment.
- **DevOps:** Automates the process of testing, building, and deploying the application.

This allows developers to deliver software updates more frequently.



2. Why Is DevOps Used?

DevOps is used to improve the speed, reliability, and efficiency of software development and deployment.

1. Faster Software Delivery

DevOps automates software building, testing, and deployment, allowing companies to release new features and bug fixes more frequently.

2. Automation

DevOps automates repetitive tasks such as:

- Building applications
- Running tests
- Deploying applications
- Monitoring servers
- Managing infrastructure

3. Continuous Integration (CI)

Developers frequently merge their code into a shared repository. Automated processes build and test the code to identify errors early.

4. Continuous Delivery and Deployment (CD)

DevOps automates the process of preparing and releasing software.

- **Continuous Delivery:** Automatically prepares code for release, usually with manual approval before production deployment.
- **Continuous Deployment:** Automatically releases changes to production when they pass the required checks.

5. Improved Collaboration

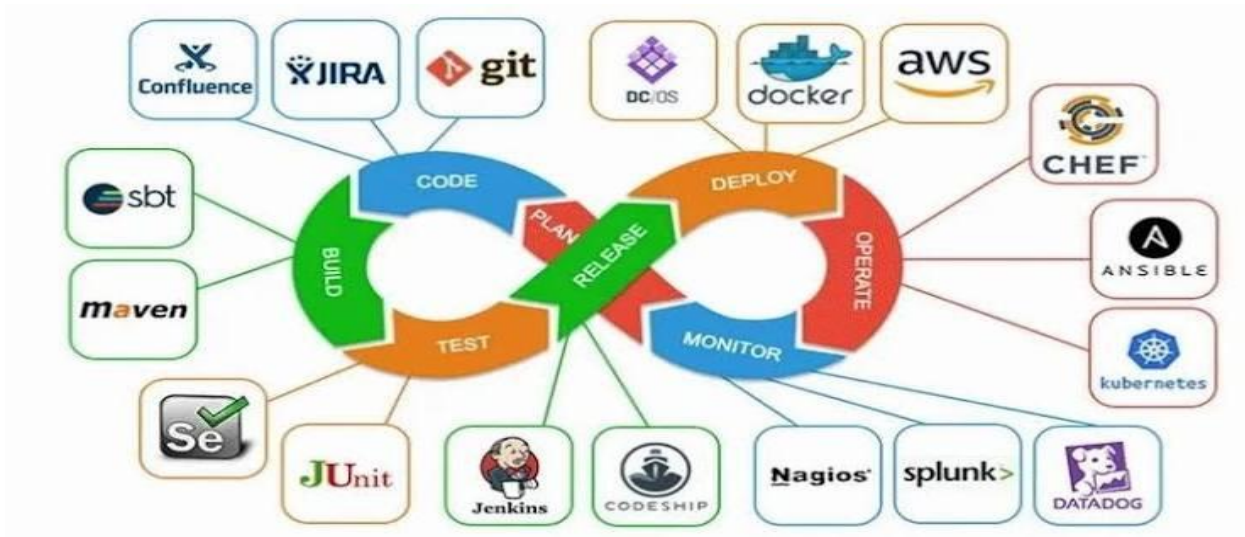
DevOps encourages developers, testers, and operations engineers to work together and share responsibility for delivering software.

6. Faster Bug Detection

Automated testing and monitoring help teams identify and resolve problems earlier.

7. Scalability

DevOps tools help companies manage applications and infrastructure as their user base and workload grow.



1. What is Docker?

Docker is an open-source platform used to build, package, distribute, and run applications inside isolated environments called containers.

Docker packages an application along with its dependencies, libraries, configuration, and runtime requirements so that it can run consistently across different environments.

For example, suppose you develop a Django application on your Windows laptop. It works perfectly on your computer, but when you run it on another computer, it may fail because of different Python versions or missing dependencies.

Docker helps solve this problem by packaging the application and its dependencies into a container.

In simple words: Docker allows you to build an application once and run it consistently on different computers and servers.

2. Why Is Docker Required?

Docker is used to solve common problems in software development and deployment.

1. Environment Consistency

Different computers may have different operating systems, software versions, and dependencies.

For example:

- Your computer has Python 3.13.
- Your friend's computer has Python 3.11.
- Your server has Python 3.12.

Your application might work on one computer but fail on another.

Docker packages the required environment with the application, reducing these differences.

2. Portability

Docker containers can run on different systems that support Docker, such as Windows, Linux, and macOS.

For example, you can develop your application on Windows and deploy it to a Linux server using Docker.

3. Isolation

Docker containers run in isolated environments.

For example, you can run two Django applications with different Python dependencies on the same machine without directly mixing their application environments.

4. Easy Deployment

Docker makes application deployment easier because the application and its dependencies are packaged together.

You can build an image on your development machine and deploy that image to a production server.

5. Scalability

Docker makes it easier to run multiple instances of an application.

For example, if your restaurant application receives more traffic, you can run additional containers to handle more requests.

Tools such as Kubernetes can help manage these containers.

6. Efficient Resource Usage

Unlike traditional virtual machines, Docker containers share the host operating system's kernel.

This generally makes containers faster to start and more resource-efficient than full virtual machines.

7. Simplified Development

Developers can use Docker to run databases, backend services, and other dependencies without installing all of them directly on their computers.

For example, you can run PostgreSQL in a Docker container.

3. What Is a Docker Image?

A Docker image is a read-only template that contains the application, its dependencies, libraries, and instructions required to create a Docker container.

An image is used to create containers.

For example, suppose you have a Django application that requires:

- Python 3.13
- Django
- PostgreSQL client libraries
- Application source code
- Required Python packages

You can create a Docker image containing the necessary application files and dependencies.

You can then use that image to create one or more containers.

Important characteristics of Docker images

1. **Read-only:** An image consists of read-only layers.
2. **Reusable:** The same image can be used to create multiple containers.
3. **Portable:** Images can be transferred between systems using a registry or other distribution methods.
4. **Layered:** Images are built from multiple layers, which can be reused.
5. **Versioned:** Images can have tags, such as `python:3.13` or `nginx:latest`.

Example

```
docker pull python:3.13
```

This command downloads the Python 3.13 image from Docker Hub.

To list downloaded images:

```
docker images
```

Example output:

REPOSITORY	TAG	IMAGE ID	SIZE
python	3.13	abc123	1.02GB
nginx	latest	def456	192MB

The image ID and sizes shown here are illustrative.

4. What Is a Docker Container?

A Docker container is an isolated, runnable instance of a Docker image.

A container includes the application and its runtime environment. It uses the image as its starting point and adds a writable layer for changes made while it runs.

For example, if you have a Python image, you can create a container from it and run a Python application inside that container.

Important characteristics of containers

1. **Isolated:** Containers have isolated processes, filesystems, and network configurations.
2. **Lightweight:** Containers share the host operating system's kernel.
3. **Portable:** Containers can run on compatible systems that support Docker.
4. **Temporary:** Containers can be stopped and removed without deleting the original image.
5. **Runnable:** Containers can execute applications and services.

Example

To create and run a container from the Python image:

```
docker run -it python:3.13
```

This starts an interactive Python environment inside a container.

To see running containers:

```
docker ps
```

To see all containers, including stopped ones:

```
docker ps -a
```

Docker Commands

No.	Docker Command	Description
1	<code>docker --version</code>	Displays the installed Docker version.
2	<code>docker pull nginx</code>	Downloads the Nginx image from Docker Hub.
3	<code>docker images</code>	Lists all locally available Docker images.
4	<code>docker build -t myapp .</code>	Builds a Docker image using a Dockerfile.
5	<code>docker run nginx</code>	Creates and starts a container using the Nginx image.
6	<code>docker run -d nginx</code>	Runs a container in the background.
7	<code>docker run -d -p 8080:80 nginx</code>	Runs Nginx and maps host port 8080 to container port 80.
8	<code>docker ps</code>	Displays all running containers.
9	<code>docker ps -a</code>	Displays all containers, including stopped containers.
10	<code>docker stop container_name</code>	Stops a running container.
11	<code>docker start container_name</code>	Starts a stopped container.
12	<code>docker restart container_name</code>	Restarts a container.
13	<code>docker rm container_name</code>	Removes a stopped container.
14	<code>docker rmi image_name</code>	Removes a Docker image.
15	<code>docker logs container_name</code>	Displays the logs of a container.
16	<code>docker exec -it container_name bash</code>	Opens an interactive Bash shell inside a running container.
17	<code>docker inspect container_name</code>	Displays detailed information about a container.
18	<code>docker volume ls</code>	Lists all Docker volumes.
19	<code>docker network ls</code>	Lists all Docker networks.
20	<code>docker compose up -d</code>	Builds (if needed) and starts services defined in a Compose file in the background.